

Spare — Project Document

Last Updated: April 2026 **Status:** Portfolio version near-complete · Production version in active development

1. Project Overview

Spare is a mobile app that connects to your bank account and automatically tracks your spending, categorizes your transactions, and rounds up purchases to save spare change.

Built for individuals who want a clean, automatic view of their finances without manually entering data or navigating complicated financial tools.

Version	Purpose	Status
Portfolio	Resume showcase, recruiter review	Near-complete. Development winding down.
Production	Real-world deployment	Actively developed. Security and infrastructure focus.

2. Current Features

Login & Signup Create an account or log in with email and password. Sessions persist across app restarts.

Dashboard (Home Screen)

- Displays your total balance and total amount spent
- Shows how much spare change you've saved through round-ups
- Spending breakdown by category with bar chart visualization

- Last 5 transactions at a glance

Connect Your Bank Tap “Connect Bank Account” to securely link a real bank account through Plaid. The bank selection UI opens natively and returns a confirmation on success.

Transactions Screen View your full transaction history in a scrollable list. Handles empty states gracefully for new users.

Tab Navigation Three-tab layout: Home, Transactions, and Insights.

Dark Mode UI Consistent dark theme across all screens.

3. In Progress

Completing the Plaid bank link flow The app can open the bank selection UI and receive a confirmation token. The remaining step is sending that token to the backend to finalize the connection and start pulling real transactions.

Fixing the dashboard display

- Balance card formatting needs to match standard currency layout
- Spending category colors aren't rendering correctly
- Confirming the “Recent Transactions” section always shows exactly 5 items

Unifying the color scheme Some screens have a light grey background that clashes with the dark card components. Needs to be made consistent throughout.

4. Upcoming / Planned

Live Bank Transactions Once the Plaid connection is fully wired, real transactions from your bank will automatically appear in the app.

Better Transaction Views Choose how you want to view your transactions:

- Grouped by category (with expandable lists)
- Full chronological list

- Search and filter by date

Search Search your transactions directly from the Transactions screen.

Charts & Insights A dedicated Insights page with pie charts, bar charts, and spending trends over time.

Shared Accounts Split finances with a partner or family. Shared pools, savings goals, and shared transaction history.

Light / Dark Mode Toggle Switch between light and dark themes from within the app.

Payment Calendar See upcoming bills and payment due dates in one place.

Credit Score View your credit score directly in the app.

5. Recent Updates

April 2026

Added:

- `syncPlaidTransactions` mutation — fetches and saves real bank transactions from Plaid
- `exchangePublicToken` mutation — exchanges the Plaid link token for a permanent access token
- `PlaidConnection` database model to store bank credentials per user
- Spare Change tracker on the dashboard (round-up savings total)
- Empty state component for screens with no data
- Bottom tab navigation (Home · Transactions · Insights)
- Dedicated Transactions screen with full transaction history

Updated:

- Migrated from REST to GraphQL API (Apollo Server)
- Migrated database to PostgreSQL + Prisma ORM
- Monorepo restructure — client and server now co-located

- Dashboard financial calculations moved to `useMemo` for performance
- Spending by category now shows percentage-based bar charts

Refactored:

- All financial logic (round-up, category aggregation, currency formatting) extracted into `financeUtils.ts`
- Firebase UID now used as the PostgreSQL user ID directly
- Plaid credentials isolated into a separate `PlaidConnection` table

Fixed:

- Firebase session now persists correctly across app restarts using `AsyncStorage`

6. Project Structure

```

SPARE/
├── graphql-server/           # Backend API
│   ├── config/
│   │   ├── adapter.js       # Database connection
│   │   └── plaid.js         # Plaid API setup
│   ├── graphql/
│   │   ├── typeDefs.js     # API schema
│   │   └── resolvers.js    # API logic
│   ├── prisma/
│   │   └── schema.prisma    # Database models
│   └── server.js            # Server entry point
├── my-app/                  # Mobile App (React Native)
│   ├── app/
│   │   ├── login.tsx
│   │   ├── signup.tsx
│   │   └── (tabs)/
│   │       ├── index.tsx   # Home screen
│   │       ├── transactions.tsx # Transactions screen
│   │       └── insights.tsx # Insights screen
│   ├── components/
│   │   ├── BalanceCard.tsx
│   │   ├── SpendingBlock.tsx
│   │   ├── TransactionRow.tsx
│   │   ├── PlaidLinkButton.tsx
│   │   └── EmptyState.tsx
│   ├── utils/
│   │   ├── financeUtils.ts # Financial calculations
│   │   └── firebaseConfig.ts
└── package.json

```

7. How It Works

Frontend (Mobile App) Built with React Native and Expo. Runs on iOS and Android from a single codebase. Handles login, navigation, and displaying your financial data. Connects to the backend via GraphQL.

Backend (API Server) A Node.js server that handles all requests from the app. Manages user accounts, transactions, and communicates with Plaid (for bank data) and the database.

Database PostgreSQL stores all user data, transactions, and bank connection info. Prisma is used to interact with the database safely and with type checking.

Bank Connection (Plaid) Plaid is a third-party service that handles the secure connection to your bank. The app uses Plaid to open a bank selection screen and retrieve your transaction history without ever handling your banking credentials directly.

Authentication (Firebase) Firebase handles login and signup. It keeps your session active so you don't have to log in every time.

8. Environment Setup

Prerequisites

- Node.js
 - Docker (for local database)
 - Expo CLI
 - ngrok (to connect phone to local server)
 - Plaid developer account (Sandbox)
 - Firebase project
-

1. Start the Database

```
docker run --name spare-db \  
-e POSTGRES_PASSWORD=your_password \  
-e POSTGRES_DB=spare \  
-p 5432:5432 \  
-d postgres
```

2. Configure Server Environment

Create `graphql-server/.env`:

```
DATABASE_URL=postgresql://user:password@localhost:5432/spare  
PLAID_CLIENT_ID=your_client_id  
PLAID_SECRET=your_secret  
PLAID_ENV=sandbox
```

3. Start the Backend

```
cd graphql-server  
npm install  
npm run dev
```

4. Expose Server with ngrok

```
ngrok http 4000
```

Copy the generated URL.

5. Configure App Environment

Create `my-app/.env`:

```
EXPO_PUBLIC_API_URL=https://your-ngrok-url.ngrok-free.app/graphql  
EXPO_PUBLIC_FIREBASE_API_KEY=your_firebase_key
```

6. Start the App

```
cd my-app  
npm install  
npx expo start
```

Scan the QR code with Expo Go on your phone.

9. Notes / Known Issues

Active Bugs

Issue	Location	Priority
Typo in import — <code>EmpyState</code> should be <code>EmptyState</code>	<code>app/(tabs)/index.tsx</code>	High
Unused React import (<code>import { use } from "react"</code>)	<code>graphql-server/resolvers.js</code>	Low
Spending category colors not rendering correctly	<code>SpendingBlock.tsx</code>	Medium
Screen backgrounds (<code>#bdb8b8</code>) clash with dark card components	Home, Transactions screens	Medium
Balance card not formatted to standard currency layout	<code>BalanceCard.tsx</code>	Medium

Known Limitations

No API authentication enforcement The backend currently accepts requests without verifying who is sending them. This is fine for the portfolio version but must be fixed before any public deployment. Fix: add Firebase token verification middleware to the server.

Bank sync not fully connected The Plaid bank link opens and returns a token successfully, but that token isn't yet sent back to the server to complete the connection. Real transactions won't appear until this step is wired up.

No transaction pagination The app loads all transactions at once. Fine for development, but will need server-side pagination before handling real user data volumes.

Plaid credentials stored as plain text Access tokens from Plaid are saved to the database without encryption. Must be encrypted before production use.

`getAllUsers` **endpoint is open** This API query returns all users with no login check. Must be removed or locked down before any public deployment.

Production Checklist (Before Go-Live)

- ☐ Add server-side Firebase token verification
- ☐ Encrypt Plaid access tokens in the database
- ☐ Remove or lock down `getAllUsers`
- ☐ Add pagination to transaction queries
- ☐ Move database to a managed cloud service
- ☐ Set up CI/CD pipeline
- ☐ Move secrets to a secrets manager
- ☐ Replace ngrok with a real hosted server